PatchSprint: Edit-Aware Speculative Decoding for Fast LLM Code Editing

Yao Huang — [Expected degree, e.g., M.S. in Computer Science], [Year in program]

Problem Description

When a developer asks a large language model (LLM) to modify code, for example "add timeouts and error handling to these API calls while keeping the existing interface," the model usually regenerates the whole file even though most of its output is copied verbatim from the input. Autoregressive decoding produces one token per forward pass, and at small batch sizes each pass is limited by memory bandwidth rather than compute, so the user waits while the model re-emits hundreds of unchanged lines. Cursor's speculative edits [5] and OpenAI's Predicted Outputs [6] exploit this redundancy, but their designs are proprietary and their behavior under load is unreported.

PatchSprint is an open-source code-editing assistant composed of a lightweight VS Code extension and a self-hosted inference backend. The project asks: how much end-to-end editing latency can an edit-aware speculative decoding proposer remove, without changing the model's output, and for which edit types and concurrency levels does the speedup hold or disappear?

Data Summary

The primary benchmark is CanItEdit [7], which contains 105 hand-crafted Python editing problems with descriptive and lazy instructions and hidden test suites, so both speed and functional correctness can be measured. I will add 20–30 tasks from open-source Python repositories to cover realistic file lengths. At run time, the input is a source file, an optional selection, and a natural-language instruction; the output is the edited file, its diff, and the result of the project's existing tests. Target models are instruction-tuned Qwen2.5-Coder [8] at 7B (primary), 14B, and 32B, which share one tokenizer for a controlled scaling study.

Methods

In speculative decoding [1], the target model verifies k proposed tokens in one forward pass and keeps the longest prefix that matches its own greedy choices, so the output is unchanged up to floating-point effects. PatchSprint replaces the draft model with an edit-aware proposer that keeps an alignment pointer into the original file, proposes upcoming source tokens while the model reproduces code, and re-synchronizes after each edit. Unlike stateless prompt lookup [3] or acceptance-driven suffix trees [4], the draft length will also account for the measured verification cost at the current batch size. The proposer will be validated in a reference decoding loop built on Hugging Face Transformers and then integrated into vLLM [2] on an NVIDIA B200 GPU (Northeastern HPC). Baselines are standard decoding, vLLM's n-gram and suffix proposers, and diff-format generation; metrics are latency, accepted length, throughput versus concurrency, exact match with greedy decoding, and test pass rate. Expected challenges include tokenization mismatches at edit boundaries, repeated code that creates ambiguous matches, and per-request proposer state inside vLLM's batched scheduler.

Preliminary Results

vLLM's n-gram proposer is stateless, searches the entire context including the instruction, and resolves ties toward the earliest match, while its dynamic mode picks k from a hand-configured batch-size table. I then replayed speculative decoding on CanItEdit with the Qwen2.5-Coder tokenizer, treating each reference solution as the model output (assuming the model reproduces the reference and memory-bound verification). A median of 81% of output tokens are copied from the input in spans of at least 8 tokens; tokenization at edit boundaries loses only about 1.5%. At $k = 16$, an oracle that copies optimally from the context needs 8.4× fewer decoding steps; a vLLM-style n-gram proposer reaches 4.9× while verifying 1.86 tokens per output token, and an early pointer prototype with n-gram fallback reaches 5.1× at 1.75. The oracle gap and the cost of rejected drafts under load are the project's targets. [To be filled: wall-clock comparison of standard versus n-gram decoding for Qwen2.5-Coder-7B on the B200.]

References

[1] Y. Leviathan, M. Kalman, and Y. Matias. Fast Inference from Transformers via Speculative Decoding. In *Proc. ICML*, 2023.

[2] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proc. SOSP*, 2023.

[3] A. Saxena. Prompt Lookup Decoding. GitHub repository, https://github.com/apoorvumang/prompt-lookup-decoding, 2023.

[4] G. Oliaro et al. SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications. In *Proc. NeurIPS*, 2025.

[5] Cursor. Editing Files at 1000 Tokens per Second. https://cursor.com/blog/instant-apply, 2024.

[6] OpenAI. Predicted Outputs. https://developers.openai.com/api/docs/guides/predicted-outputs, accessed Oct. 2026.

[7] F. Cassano et al. Can It Edit? Evaluating the Ability of Large Language Models to Follow Code Editing Instructions. arXiv:2312.12450, 2023.

[8] B. Hui et al. Qwen2.5-Coder Technical Report. arXiv:2409.12186, 2024.